Rapport de code

SymPy

KOLA MARCEL

Analyse Numérique

11 avril 2026

I. Explication de code avec SymPy((approche symbolique)

1. on a l'initialisation des variable avec :

```
x_pts = [0, 1, 2, 3, 4]
y_pts = [1, 3, 2, 5, 4]
valeur_test = 2.5
```

2. SymPy construit l'équation mathématique exacte de la spline, sous forme d'une expression Piecewise (fonctions définies par morceaux). Le 2 désigne le degré quadratique. On peut lire et manipuler les formules.

```
x_sym = sp.Symbol('x')
spline_sympy = sp.interpolating_spline(2, x_sym, x_pts,
y_pts)
```

3. Affiche dans la console les polynômes pour chaque intervalle (ex : $ax^2 + bx + c$ si $0 \leq x < 1$, etc.), ce que SciPy ne permet pas.

```
sp.pprint(spline_sympy)
```

4. **Évaluation locale :**
   a. .subs(x_sym, 2.5) remplace symboliquement x par 2.5
   b. .evalf() convertit le résultat symbolique en nombre décimal

```
y_test_sympy = spline_sympy.subs(x_sym,
valeur_test).evalf()
```

5. **Évaluation globale (pour le graphe) :** lambdify convertit l'expression symbolique en une fonction Python rapide compatible NumPy, pour pouvoir tracer le graphe efficacement (car SymPy seul est trop lent sur 100 points).

```
f_rapide = sp.lambdify(x_sym, spline_sympy, 'numpy')
y_fine_sympy = f_rapide(x_fine)
```

II. Guide Technique : Interpolation et Évaluation avec SymPy

1. Construction

sp.interpolating_spline(d, x, x_pts, y_pts) : Elle construit une fonction Piecewise (par morceaux). Elle définit $n-1$ équations de degré d et résout les conditions de continuité et de dérivabilité aux nœuds.

L'Algorithme interne : Génération de Système Linéaire Symbolique

Cette fonction ne calcule aucun nombre flottant. Elle construit une structure logique appelée **Arbre Syntaxique Abstrait (AST)** de type Piecewise (fonction par morceaux).

1. **Génération des inconnues** : Pour *n* points, elle crée n−1 polynômes sous forme de symboles : . $a_i x^2 + b_i x + c_i$
2. **Mise en équation** : Elle génère les équations de continuité (valeurs aux nœuds et dérivées premières égales).
3. **Résolution symbolique** : Elle utilise un algorithme d'élimination de Gauss-Jordan purement algébrique pour isoler chaque coefficient a_i, b_i, c_i et les exprimer sous forme de fractions exactes.
4. **Sortie** : Elle retourne une structure conditionnelle imbriquée stricte (ex : Si x<1 alors P_1, Sinon Si...).

2. Évaluation et Calcul de Position

Pour obtenir une valeur en un point x_0, SymPy utilise un cycle en trois étapes :

1. **.subs(x, x_0) (Substitution)** : Remplace physiquement le symbole x par la valeur dans l'arbre syntaxique de l'expression. Pour une spline, c'est ici que l'algorithme « choisit » le morceau d'équation correspondant à l'intervalle de x_0.
2. **.evalf() (Évaluation)** : Convertit l'expression exacte (fractions, racines) en un nombre flottant avec une précision arbitraire.
3. **lambdify() (Vectorisation)** : Traduit l'expression SymPy en code machine NumPy. C'est l'outil qui permet de calculer la position de milliers de points (x_fine) instantanément pour tracer un graphe.

Tableau de synthèse

Concept	SymPy (Symbolique)
Nature	Expression mathématique (Texte/Logique)
Position locale	.subs(x, val).evalf()
Position globale	lambdify(x, expr)(x_fine)
Avantage	Précision exacte, lisibilité de l'équation

3. Algorithmes Détaillés

3.1 Évaluation Locale : .subs(x, val).evalf()

Algorithme : Parcours d'Arbre et Arithmétique Multiprécision

1. **.subs(x, val) (Parcours)** : L'algorithme parcourt l'arbre Piecewise. Il évalue symboliquement chaque condition booléenne (ex : 0≤2.5 et 2.5<1). Dès qu'il trouve la condition « Vraie », il extrait le polynôme symbolique correspondant et remplace le nœud *x* par 2.5
2. **.evalf() (Calcul)** : Il transmet l'expression (ex : 15/4 *·(2.5)²-…) à la bibliothèque mpmath. Celle-ci applique un algorithme d'arithmétique multiprécision

(gérant manuellement la mantisse et l'exposant) pour additionner et multiplier les valeurs sans subir les erreurs d'arrondi habituelles des processeurs 64 bits.

Sa complexité dépend de la profondeur de l'arbre, généralement O(n) pour une fonction Piecewise.

Pseudo-code : Parcours Conditionnel

```
Fonction Evaluer_Piecewise(arbre_piecewise, variable, val):
    Pour chaque (expression, condition) dans arbre_piecewise:
        // Évaluation booléenne de la condition (ex: 1 <= 2.5 < 2 est Faux)
        Si Evaluer_Condition(condition, variable, val) est VRAI:

            // Substitution purement textuelle/symbolique
            expr_substituee = Remplacer(expression, variable, val)

            // Calcul arithmétique multi-précision (équivalent de .evalf())
            Retourner Calcul_Flottant(expr_substituee)

    Retourner Erreur("Valeur hors domaine")
```

3.2 Évaluation Globale : lambdify(x, expr)(x_fine)

Algorithme : Génération et Compilation de Code à la volée

C'est un pont fascinant entre le monde symbolique et numérique.

1. **Génération de chaîne de caractères** : lambdify analyse l'expression Piecewise SymPy et écrit un script Python/NumPy sous forme de texte. Il traduit les conditions SymPy en fonctions NumPy vectorisées comme numpy.select ou numpy.piecewise.
2. **Compilation (eval)** : Il compile cette chaîne de caractères en une vraie fonction Python exécutable en mémoire.
3. **Exécution ((x_fine))** : Lorsqu'on passe le tableau x_fine, c'est cette nouvelle fonction compilée qui prend le relais. Elle exécute l'algorithme sur tous les points d'un coup.

Pseudo-code : Vectorisation (SIMD)

```
Fonction Evaluer_Vecteur(x_fine, algo_local):
    // 1. Délégation au langage C / Fortran
    // 2. Traitement par blocs (Chunks) dans le cache du processeur
    // 3. Application de l'algo_local sur plusieurs registres simultanément
    y_fine = Allouer_Memoire(longueur(x_fine))

    Instruction_Processeur_Vectorielle:
        y_fine[:] = algo_local(x_fine[:])

    Retourner y_fine
```

4. Algorithme Complet de sp.interpolating_spline

Contrairement à SciPy qui manipule des tableaux de nombres, sp.interpolating_spline est un **Générateur et Solveur de Système Algébrique**. Son but est de trouver les expressions exactes (avec des fractions parfaites) pour chaque segment de la spline.

Pour *n* points, il y a n−1 segments. Si on demande un degré *d*=2 (quadratique), chaque segment possède 3 inconnues (a_i, b_i, c_i). SymPy doit donc créer et résoudre un système de 3(n−1) équations.

L'algorithme se déroule en **5 grandes étapes internes** :

1. **Génération des inconnues symboliques** : SymPy crée des variables mathématiques abstraites pour les coefficients de chaque segment.
2. **Pose des conditions d'interpolation** : Il écrit les équations forçant chaque segment de parabole à passer par ses points de début et de fin.
3. **Pose des conditions de raccordement (Lissage)** : Il calcule la dérivée symbolique de chaque segment ($2a_i x + b_i$) et pose les équations forçant la dérivée à être égale entre deux segments adjacents.
4. **Résolution matricielle exacte** : Il place toutes ces équations dans une grande matrice et utilise l'algorithme du **Pivot de Gauss symbolique** pour isoler chaque coefficient a_i, b_i, c_i.
5. **Assemblage Piecewise** : Il construit la structure conditionnelle finale (Si *x* est dans tel intervalle, alors utiliser telle équation résolue).

Pseudo-code complet

```
Fonction Algorithme_Interpolating_Spline(degre, variable_x, x_pts, y_pts):
    n = longueur(x_pts)
    nb_segments = n - 1
    equations = Liste_Vide()
    symboles = Liste_Vide()
    segments_polynomiaux = Liste_Vide()

    // 1. Création des symboles et polynômes génériques
    Pour i de 0 à nb_segments - 1:
        a, b, c = Nouveaux_Symboles(3)
        Ajouter (a, b, c) à symboles
        P = a * variable_x^2 + b * variable_x + c
        Ajouter P à segments_polynomiaux

    // 2. Conditions d'interpolation (Passage par les points)
    Pour i de 0 à nb_segments - 1:
        P = segments_polynomiaux[i]
        Ajouter_Equation(equations, P(x_pts[i]) == y_pts[i])
        Ajouter_Equation(equations, P(x_pts[i+1]) == y_pts[i+1])

    // 3. Conditions de continuité (Dérivées égales aux nœuds internes)
    Pour i de 0 à nb_segments - 2:
        P_actuel = segments_polynomiaux[i]
        P_suivant = segments_polynomiaux[i+1]
        Derivee_actuelle = Deriver(P_actuel, variable_x)
        Derivee_suivante = Deriver(P_suivant, variable_x)
        Ajouter_Equation(equations, Derivee_actuelle(x_pts[i+1]) == Derivee_suivante(x_pts[i+1]))

    // 4. Condition aux limites (pour fermer le système)
    Derivee_0 = Deriver(segments_polynomiaux[0], variable_x)
    Ajouter_Equation(equations, Derivee_0(x_pts[0]) == 0)

    // 5. Résolution du système linéaire symbolique (Pivot de Gauss exact)
    Solutions_Exactes = Solveur_Symbolique_SymPy(equations, symboles)

    // 6. Construction de l'objet Piecewise
    Objet_Piecewise = Nouveau_Piecewise()
    Pour i de 0 à nb_segments - 1:
        Polynome_Resolu = Substituer(segments_polynomiaux[i], Solutions_Exactes)
        Condition_Intervalle = (x_pts[i] <= variable_x) ET (variable_x < x_pts[i+1])
        Ajouter_Branche(Objet_Piecewise, Polynome_Resolu, Condition_Intervalle)

    Retourner Objet_Piecewise
```